Laporan Tugas Besar 1

IF3170 Inteligensi Artifisial

Optimasi Pengisian Paket pada Truk Peti Kemas dengan Algoritma Local Search

Tahun Akademik 2026–2027



Disusun oleh:

Kelompok 37

Samuelson Dharmawan Tanuraharja – 13524001
Nicholas Wise Saragih Sumbayak – 13524037
Neswa Eka Anggara – 13524136

Laboratorium Inteligensi Artifisial
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1 Pendahuluan **2**
1.1 Latar Belakang . . . 2
1.2 Rumusan Persoalan dan Tujuan . . . 3
1.3 Ruang Lingkup Persoalan . . . 3

2 Dasar Teori **4**
2.1 Optimasi dan *Local Search* . . . 4
2.2 *State*, *Neighbor*, dan *Search Landscape* . . . 4
2.3 *Hill Climbing* . . . 5
2.4 *Simulated Annealing* . . . 6
2.5 *Genetic Algorithm* . . . 7

3 Pembahasan dan Implementasi **8**
3.1 Arsitektur Aplikasi . . . 8
3.1.1 Struktur Aplikasi . . . 8
3.1.2 Tech Stack . . . 9
3.2 Pemodelan Problem dan Constraint . . . 9
3.2.1 Data Problem . . . 9
3.2.2 Representasi State dan Orientasi . . . 10
3.2.3 Validasi Constraint . . . 11
3.2.4 Objective Function . . . 13
3.3 Implementasi Algoritma *Local Search* . . . 13
3.3.1 Inisialisasi State dan Pembangkitan Neighbor . . . 13
3.3.2 Hill Climbing . . . 14
3.3.3 Simulated Annealing . . . 17
3.3.4 Genetic Algorithm . . . 17

4 Eksperimen dan Analisis **19**
4.1 Rancangan Eksperimen . . . 19
4.1.1 Data Uji dan Lingkungan Eksperimen . . . 19
4.1.2 Konfigurasi Parameter dan Pengulangan Eksperimen . . . 19
4.2 Hasil Eksperimen dan Visualisasi Program . . . 20
4.2.1 Hill Climbing . . . 20
4.2.2 Simulated Annealing . . . 20
4.2.3 Genetic Algorithm . . . 21
4.3 Analisis Eksperimen . . . 21
4.3.1 Perbandingan Kualitas Solusi . . . 21
4.3.2 Perbandingan Durasi Pencarian . . . 21
4.3.3 Konsistensi Hasil Eksperimen . . . 21
4.3.4 Pengaruh Parameter Genetic Algorithm . . . 21

5 Kesimpulan dan Saran **22**
5.1 Kesimpulan . . . 22
5.2 Saran . . . 22

6 Pembagian Tugas Tiap Anggota Kelompok **22**

Daftar Pustaka **22**

A Form Penggunaan Generative AI **22**

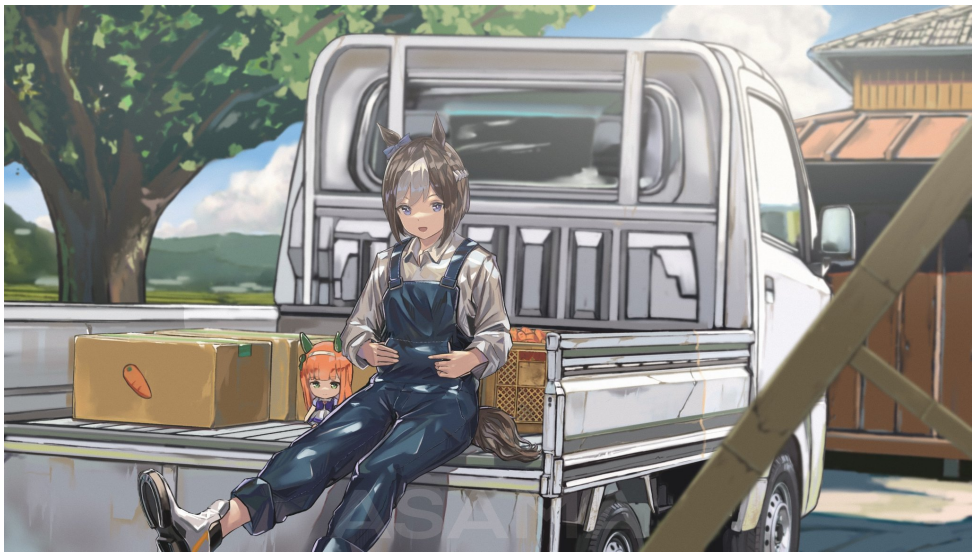
B Lampiran Lainnya (Bila Ada) **22**

1 Pendahuluan

1.1 Latar Belakang

Saat menyiapkan pengiriman Mejiro Express, Special Week menghadapi paket dengan ukuran, nilai, berat, dan tingkat kerapuhan yang berbeda-beda. Sebagai pengemudi, ia perlu menentukan paket yang dapat dibawa sekaligus susunannya di dalam truk peti kemas. Paket bernilai tinggi tentu menarik untuk dimuat, tetapi ruang dan kapasitas angkut truk membatasi pilihan tersebut. Paket yang sudah masuk pun perlu ditata agar tidak bertumpang tindih atau melayang, sementara paket rapuh tidak dapat menjadi penopang paket lain [1].

Keputusan tersebut saling berkaitan. Memilih satu paket mengubah ruang tersisa dan berat muatan, sementara posisi serta orientasi paket menentukan apakah paket lain masih dapat ditempatkan dengan aman. Karena itu, program menilai susunan lengkap, termasuk paket yang dibiarkan di luar peti kemas, alih-alih memilih paket satu per satu berdasarkan nilainya saja. Banyaknya kemungkinan susunan membuat pemeriksaan seluruh kombinasi tidak praktis; pencarian lokal menjadi cara untuk memperbaiki kandidat melalui perubahan bertahap.



Gambar 1. Mejiro Express sebagaimana ditunjukkan dalam spesifikasi
(Sumber: 真夏の银堡垒🐴C108土曜日 東地区 ホ-10a on X).

1.2 Rumusan Persoalan dan Tujuan

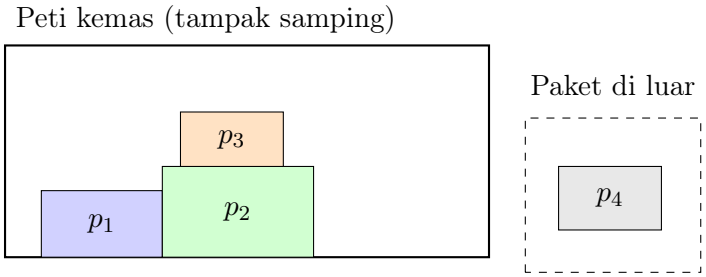
Masukannya berupa paket berbentuk balok atau kubus serta peti kemas dengan ukuran dan kapasitas angkut tertentu. Program memilih paket yang dimuat, lalu menentukan posisi dan orientasi masing-masing. Pilihan memuat paket bernilai tinggi dapat mengurangi ruang atau kapasitas bagi paket lain, sehingga setiap keputusan dinilai sebagai bagian dari satu susunan. Tujuannya memaksimalkan ***objective function*** (fungsi objektif), yaitu total nilai paket dalam susunan yang sah. Paket yang tidak dimuat tetap tercatat di luar peti kemas [1].

Pekerjaan ini menerapkan empat varian *hill climbing* (HC), *simulated annealing* (SA), dan *genetic algorithm* (GA). State awal, state akhir, state terbaik, skor, riwayat pencarian, dan waktu eksekusi menjadi informasi pembanding kualitas dan durasi pencarian. Seed berbeda pada pengulangan eksperimen memungkinkan variasi hasil pencarian acak diamati [1].

1.3 Ruang Lingkup Persoalan

Posisi paket dinyatakan dengan koordinat bilangan bulat dalam ruang tiga dimensi, dan orientasinya dapat diubah melalui rotasi 90° terhadap sumbu ruang. Setiap paket yang dimuat harus berada di dalam peti kemas, tidak bertumpang tindih, dan memiliki penopang. Paket rapuh tidak boleh menopang paket lain, sedangkan total berat muatan harus tetap dalam kapasitas truk. Paket yang tidak dimuat berada sepenuhnya di luar peti kemas [1]. Batasan ini membuat perubahan susunan perlu diperiksa sebagai satu kesatuan.

Implementasi mendukung satu atau beberapa truk dalam satu problem. Setiap paket dapat ditugaskan kepada paling banyak satu truk, dan nilai muatan dijumlahkan pada seluruh armada. Selain objektif utama berupa total nilai, tersedia objektif tambahan yang membobot nilai berdasarkan ETA paket, sehingga nilai dan waktu kedatangan dapat dipertimbangkan bersama. Kedua mode memakai aturan kelayakan yang sama; dalam ruang kandidat yang besar ini, pencarian lokal menelusuri perubahan susunan tanpa mengabaikan batas ruang, berat, dan penopang.



Gambar 2. Ilustrasi konseptual paket termuat dan paket di luar peti kemas. Susunan sebenarnya ditentukan dalam tiga dimensi.

2 Dasar Teori

2.1 Optimasi dan *Local Search*

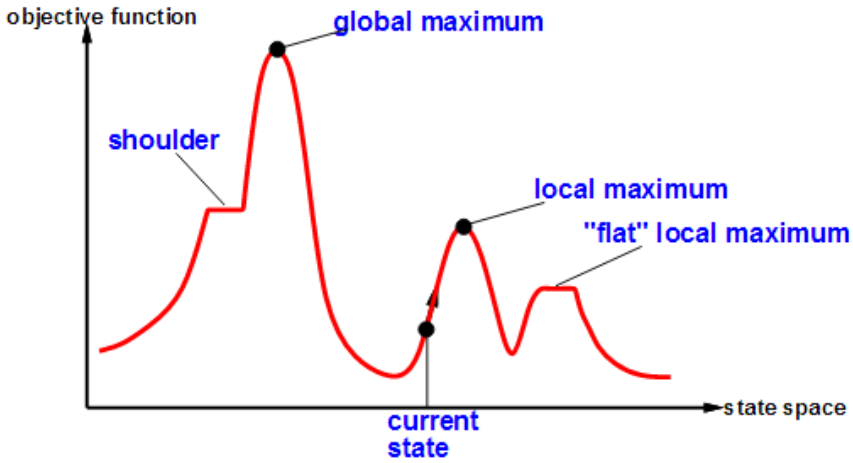
Dalam optimasi, *objective function* memberi nilai pada setiap kandidat solusi, dan pencarian diarahkan pada kandidat bernilai tinggi. Jika terdapat ***constraint*** (batasan), kandidat yang dipertimbangkan termasuk dalam himpunan yang memenuhi batasan tersebut. Dengan $\mathcal{F}$ sebagai himpunan kandidat yang sah, tujuan umum masalah maksimisasi dapat ditulis $\max_{s \in \mathcal{F}} f(s)$, dengan $f(s)$ sebagai nilai kandidat s [2].

Local search bekerja pada satu kandidat lengkap dan memperbaikinya melalui perubahan bertahap. Nilai solusi ditentukan oleh konfigurasi akhir, bukan urutan perubahan yang membentuknya. Pendekatan ini sesuai ketika ruang kemungkinan terlalu besar untuk diperiksa satu per satu [2, 3].

2.2 *State*, *Neighbor*, dan *Search Landscape*

State (keadaan) adalah kandidat solusi yang lengkap. Pencarian berawal dari *initial state* (state awal), yang dapat dibangkitkan secara acak. Sebuah *move* (langkah perubahan) menghasilkan ***neighbor*** (tetangga), yaitu state yang berbeda pada sebagian komponennya. Kumpulan neighbor yang dapat dicapai menentukan arah perpindahan dari state saat ini [3].

Nilai setiap state dapat dibayangkan sebagai ketinggian dalam *search landscape* (lanskap pencarian). ***Local maximum*** (puncak lokal) bernilai setidaknya sama tinggi dengan seluruh neighbor-nya, walaupun state lain mungkin memiliki nilai lebih tinggi. Nilai tertinggi di seluruh ruang pencarian disebut *global maximum* (puncak global). *Plateau* (daerah datar) merupakan kumpulan state bertetangga dengan nilai yang sama. Puncak lokal dan plateau menjelaskan mengapa pencarian yang hanya menerima kenaikan nilai dapat berhenti sebelum mencapai nilai tertinggi keseluruhan [2, 3].



Gambar 3. *Search landscape* dengan global maximum, local maximum, dan plateau. Diambil dari salindia *Beyond Classical Search* [3].

2.3 *Hill Climbing*

Seperti terlihat pada Gambar 3, pencarian yang hanya memilih kenaikan nilai dapat berhenti di local maximum. ***Hill climbing*** memulai dari satu state, membandingkannya dengan neighbor, lalu berpindah menurut aturan varian yang digunakan. Pada *steepest-ascent hill climbing*, versi teoretis mengevaluasi seluruh neighbor dan memilih yang memiliki nilai tertinggi, selama nilainya melebihi state saat ini. Jika beberapa neighbor sama-sama terbaik, pemilihan salah satunya tidak mengubah nilai yang dicapai pada langkah itu.

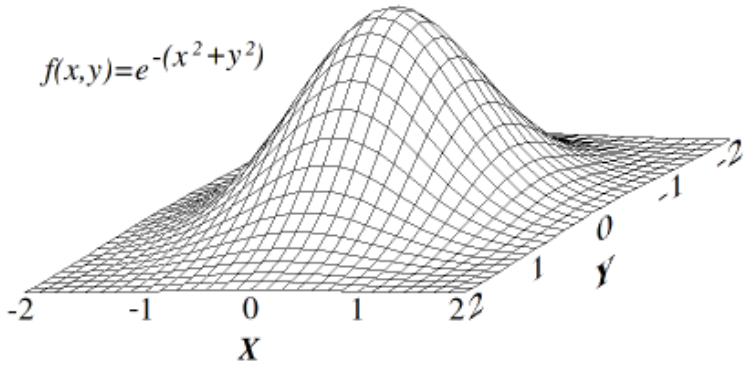
Dalam implementasi, seluruh neighborhood tidak diperiksa. Algoritma membangkitkan sampel terbatas neighbor yang sah, lalu memilih kandidat bernilai tertinggi dari sampel tersebut. Karena itu, **pemeriksaan terbatas pada sampel** berarti tidak ditemukannya peningkatan hanya menunjukkan bahwa sampel saat itu tidak memuat kandidat yang lebih baik; keadaan ini belum memastikan semua neighbor sudah diperiksa. Steepest-ascent menghindari langkah yang menurunkan atau mempertahankan skor, sehingga pencarian dapat berakhir di puncak lokal atau berhenti saat sampelnya tidak menunjukkan kenaikan [2, 3].

Stochastic hill climbing memilih secara acak dari neighbor yang meningkatkan skor. Pemilihan seragam memberi setiap kandidat peningkat peluang yang sama, sedangkan skema berbobot dapat memberi peluang lebih besar kepada kenaikan yang lebih besar. Implementasi menggunakan **pemilihan seragam**, bukan pembobotan berdasarkan besar kenaikan. Cara ini dapat mengambil langkah yang tidak terbaik pada sampel, tetapi tetap menuntut peningkatan ketat; akibatnya, pencarian berhenti ketika sampel tidak menyediakan peningkat dan tidak bergerak melintasi plateau [2, 3].

Hill climbing with sideways move juga menerima neighbor yang skornya sama, sehingga dapat bergerak di antara state pada plateau dan mungkin mencapai state yang memiliki peningkatan di sisi lain. Langkah setara tidak menaikkan skor, dan tanpa batas dapat membuat pencarian berputar atau bergerak lama tanpa kemajuan. Karena itu, **jumlah langkah setara berturut-turut dibatasi**. Langkah yang meningkatkan skor mengatur ulang hitungan tersebut; setelah batas tercapai, langkah setara berikutnya ditolak, sementara peningkatan ketat tetap dapat diterima.

Random-restart hill climbing mengulang steepest-ascent dari state awal berbeda untuk mencoba wilayah pencarian lain. Implementasi menjalankan satu climb dari state awal yang diberikan, lalu melakukan 10 restart, sehingga terdapat 11 climb. Setiap climb memiliki anggaran iterasinya sendiri; total pekerjaan dapat lebih besar daripada satu climb, tetapi jumlah restart dan sampel tetap membatasi bagian ruang yang diperiksa. Hasil terbaik dipilih dari seluruh climb, sedangkan state akhir adalah hasil climb terakhir. Restart dapat mengurangi ketergantungan pada satu puncak lokal, tetapi tidak memastikan optimum global; pencarian tetap tidak mengetahui solusi terbaik yang belum dikunjungi [2, 3].

Gambar 4 memperlihatkan fungsi dua variabel dengan satu puncak. Pada masalah maksimisasi, perpindahan ke nilai yang lebih tinggi dapat dibayangkan sebagai langkah menaiki permukaan tersebut. Susunan paket merupakan state diskret dengan banyak constraint, sehingga lanskapnya tidak harus sehalus ilustrasi ini; puncak lokal dan plateau tetap menjadi hambatan seperti pada Gambar 3.



Gambar 4. Ilustrasi permukaan objektif untuk hill climbing. Sumber: Wikimedia Commons, *Hill climb.png*.

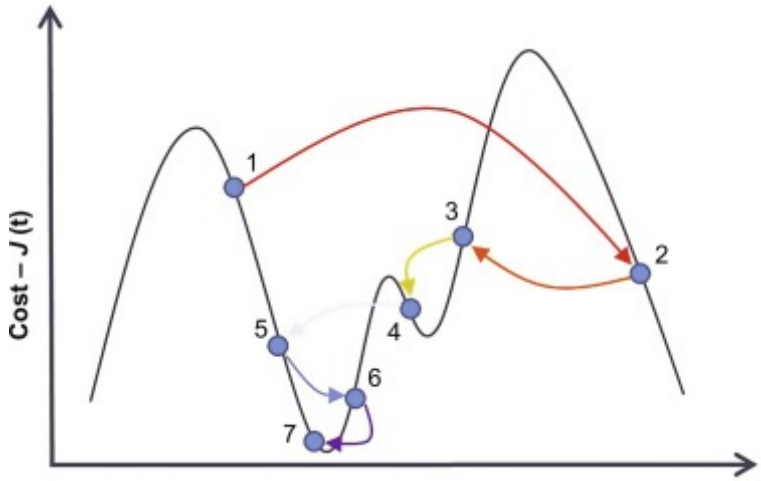
2.4 *Simulated Annealing*

Hill climbing tidak selalu bisa keluar dari local maximum dengan move yang hanya memperbaiki nilai. ***Simulated annealing*** sesekali menerima move yang menurunkan nilai agar pencarian dapat berpindah ke daerah lain. Peluang menerima penurunan ditentukan oleh *temperature* (temperatur). Untuk $\Delta f = f(s') - f(s) < 0$ dan $T > 0$, peluang menerima neighbor s' adalah

$$\Pr(\text{accept } s') = \exp\left(\frac{\Delta f}{T}\right). \tag{1}$$

Pada temperature tinggi, algoritma lebih sering menerima penurunan nilai sehingga dapat keluar dari local maximum. Seiring temperature turun, peluang itu mengecil. *Cooling schedule* (jadwal penurunan temperatur) menentukan seberapa cepat perilaku pencarian berubah. Jadwal yang terlalu cepat dapat membuat pencarian kembali terjebak sebelum sempat menjelajahi state lain [2, 3].

Gambar 5 menunjukkan perpindahan pada lanskap biaya, termasuk langkah yang sementara menaikkan biaya sebelum pencarian mencapai daerah yang lebih rendah. Ilustrasi ini memakai sudut pandang minimisasi. Untuk persoalan pemuatan yang memaksimalkan $f(s)$, biaya dapat dinyatakan sebagai $E(s) = -f(s)$, sehingga kenaikan biaya pada gambar setara dengan penurunan skor objektif yang sesekali diterima oleh SA.

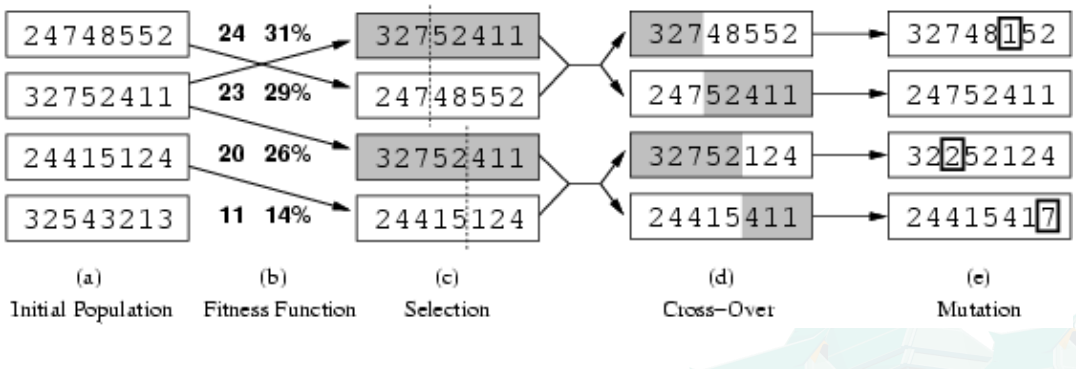


Gambar 5. Ilustrasi perpindahan simulated annealing pada lanskap biaya. Sumber: Elsevier, ilustrasi 2.14.

2.5 *Genetic Algorithm*

Jika hill climbing dan simulated annealing mengubah satu state dalam satu lintasan pencarian, ***genetic algorithm*** bekerja dengan *population* (populasi) yang berisi sejumlah *individual* (kandidat solusi). Setiap individual dinilai memakai *fitness function* (fungsi penilaian). Melalui *selection* (pemilihan induk), individual yang lebih baik mendapat peluang lebih besar untuk menghasilkan keturunan [2, 3].

Generasi baru dibentuk melalui *crossover* (penggabungan bagian dari dua induk) dan *mutation* (perubahan acak pada individual). Crossover mewariskan bagian dari solusi yang sudah ada, sementara mutation membantu menjaga keragaman population. Ukuran population dan banyaknya *generation* (generasi) memengaruhi jumlah kandidat yang diperiksa serta waktu pencarian. Bila suatu operator menghasilkan kandidat yang melanggar constraint, kandidat itu perlu ditangani sebelum dipakai sebagai solusi [2, 3].



Gambar 6. Alur selection, crossover, dan mutation pada genetic algorithm. Diambil dari salindia *Beyond Classical Search* [3].

3 Pembahasan dan Implementasi

3.1 Arsitektur Aplikasi

Pengguna mengubah paket dan truk melalui formulir web yang dimulai dari fixture armada bawaan di `src/frontend/src/mocks/fleetFixture.ts`. Belum ada alur impor berkas problem.

Konfigurasi diperiksa melalui endpoint `POST /api/problems/validate`. Endpoint ini memvalidasi model dan batas masukan, bukan susunan paket. Ketika run dimulai, frontend memeriksa parameter lalu mengirim problem, algoritma, seed, objektif, parameter, dan pilihan riwayat ke `POST /api/experiments/run`. Backend memvalidasi skema dan anggaran eksekusi, membangkitkan state awal, menjalankan algoritma, lalu memeriksa kelayakan state awal, akhir, dan terbaik serta kecocokan skornya sebelum membentuk respons.

Run dari UI memakai `persist=false`. Hasil yang disimpan dapat dimuat kembali melalui `GET/api/experiments`. Endpoint preset GA dan batch tersedia melalui `GET/api/experiments/presets/genetic_algorithm` dan `POST/api/experiments/batch`. Problem yang sedang diedit untuk run berikutnya terpisah dari problem asli milik run yang sedang ditampilkan.

3.1.1 Struktur Aplikasi

Folder utama aplikasi tersusun sebagai berikut. Pohon ini hanya menampilkan direktori yang berperan dalam alur pencarian dan visualisasi.

```
.
|-- data/
|-- output/
|-- src/
    |-- backend/
    | |-- app/
    | |-- api/
    | |-- domain/
    | |-- experiments/
    | |-- optimization/
    |-- frontend/
        |-- src/
            |-- api/
            |-- components/
            |-- stores/
```

Di backend, `domain` mendefinisikan data, `optimization` memuat validator dan algoritma, sedangkan `experiments` mengatur eksekusi, ringkasan, dan pencatatan hasil. Di frontend, `components` memuat form, grafik, inspector, dan tampilan truk; `api` menangani request serta pemeriksaan respons, dan `stores` menyimpan run terpilih serta indeks riwayat. Fixture awal berada di `src/frontend/src/mocks`; `data/` belum memuat dataset. Jika persistensi diaktifkan, run tunggal disimpan sebagai JSON di `output/experiments/`, sedangkan batch menyimpan JSON dan CSV di `output/batches/`. Berkas-berkas tersebut merupakan keluaran yang dapat dibuat, bukan hasil yang telah tersedia di repositori saat ini.

3.1.2 Tech Stack

Backend ditulis dengan **Python 3.12**. **FastAPI** menyediakan endpoint untuk memvalidasi problem, menjalankan eksperimen tunggal atau batch, dan membaca hasil yang tersimpan. Model **Pydantic** memeriksa tipe serta rentang nilai saat request diterima. Sesudah itu, modul optimasi menjalankan aturan yang sama untuk semua algoritma. **NumPy** digunakan untuk occupancy grid tiga dimensi, sehingga pemeriksaan ruang terisi dapat dilakukan pada irisan sel paket.

Frontend dibangun dengan **React** dan **TypeScript**. Komponen React memisahkan form problem, kontrol pencarian, visualisasi, dan ringkasan hasil. TypeScript membantu menjaga bentuk request dan respons tetap sesuai kontrak backend. **Vite** dipakai untuk menjalankan frontend selama pengembangan dan menghasilkan berkas aplikasi saat build.

Untuk menampilkan hasil, **React Three Fiber** dan **Three.js** menggambar peti kemas serta posisi paket dalam ruang tiga dimensi. **Recharts** menampilkan perubahan skor selama iterasi. **Zustand** menyimpan eksperimen yang dipilih dan indeks iterasi untuk replay.

3.2 Pemodelan Problem dan Constraint

Pemodelan dimulai dari data paket dan truk. Dari data itu, program membentuk state yang mencatat posisi setiap paket. State lalu melewati pemeriksaan constraint sebelum objective dihitung. Urutan ini dipakai bersama oleh seluruh algoritma, sehingga nilai yang dibandingkan selalu berasal dari **susunan yang sah**.

3.2.1 Data Problem

Kelas `Problem` menyimpan dua daftar: `PackageSpec` dan `TruckSpec`. Setiap paket memiliki ID, dimensi lebar, panjang, dan tinggi, kemudian `value`, `weight`, `is_fragile`, serta `eta`. Truk menyimpan ID, tiga dimensi ruang muat, dan kapasitas maksimum. Cuplikan 1 hingga 3 memperlihatkan hubungan ketiga kelas tersebut.

Cuplikan 1. Atribut paket pada `PackageSpec`

```python
class PackageSpec(BaseModel):
    model_config = ConfigDict(frozen=True, allow_inf_nan=False)

    id: str
    width: int = Field(gt=0)
    length: int = Field(gt=0)
    height: int = Field(gt=0)
    value: float = Field(ge=0)
    weight: float = Field(ge=0)
    is_fragile: bool
    eta: int = Field(ge=0)
```

Validasi masukan dilakukan sejak model dibuat. Dimensi harus berupa bilangan bulat positif, sedangkan nilai, berat, ETA, dan kapasitas tidak boleh negatif maupun tak hingga. `Problem` menolak ID paket atau truk yang berulang dan mensyaratkan setidaknya satu truk. Karena `trucks` berupa daftar, struktur yang sama dapat dipakai untuk satu truk maupun beberapa truk. Setelah data ini tersedia, penempatan setiap paket dicatat dalam state.

Cuplikan 2. Atribut truk pada `TruckSpec`

```python
class TruckSpec(BaseModel):
    model_config = ConfigDict(frozen=True, allow_inf_nan=False)

    id: str
    width: int = Field(gt=0)
    length: int = Field(gt=0)
    height: int = Field(gt=0)
    max_capacity: float = Field(ge=0)
```

Cuplikan 3. Daftar paket dan truk dalam `Problem`

```python
class Problem(BaseModel):
    packages: list[PackageSpec]
    trucks: list[TruckSpec]

    @model_validator(mode="after")
    def validate_ids_and_trucks(self) -> "Problem":
        if not self.trucks:
            raise ValueError("at least one truck is required")
        if len({package.id for package in self.packages}) != len(self.packages):
            raise ValueError("package IDs must be unique")
        if len({truck.id for truck in self.trucks}) != len(self.trucks):
            raise ValueError("truck IDs must be unique")
        return self
```

3.2.2 Representasi State dan Orientasi

Kelas `State` berisi daftar `PackagePlacement`. Pada state yang sah, **setiap paket memiliki tepat satu placement**. Setiap placement memuat ID paket dan truk, `Position`, serta `Orientation`. Koordinat `Position` menunjuk sudut kiri-bawah-belakang paket. Sumbu x mengikuti lebar truk, y mengikuti panjang, dan z mengikuti tinggi. Paket di luar truk memiliki `truck_id` dan `position` bernilai `None`; untuk paket yang dimuat, keduanya berisi identitas truk dan posisi, sesuai pemeriksaan `validate_assignment` pada Cuplikan 4.

Cuplikan 4. Model penempatan paket

```python
class PackagePlacement(BaseModel):
    package_id: str
    truck_id: str | None = None
    position: Position | None = None
    orientation: Orientation = Orientation.WLH

    @model_validator(mode="after")
    def validate_assignment(self) -> "PackagePlacement":
        if (self.truck_id is None) != (self.position is None):
            raise ValueError("truck_id and position must both be set or both be None")
        return self
```

Enam pilihan `Orientation` didefinisikan pada `app/domain/package.py`. Fungsi `get_oriented_dimensions` mengubah orientasi menjadi dimensi efektif (w_i, l_i, h_i) tanpa men-

gubah dimensi asli paket. Untuk kubus atau balok dengan sisi yang sama, beberapa orientasi menghasilkan dimensi efektif identik. Fungsi `get_unique_orientations` menghapus pilihan duplikat tersebut saat orientasi akan diacak. State yang terbentuk kemudian diperiksa sebagai satu kesatuan, bukan per paket saja.

3.2.3 Validasi Constraint

Fungsi `validate_state` pada `app/optimization/constraints.py` menjadi **pemeriksa akhir setiap kandidat**. Pemeriksaan dimulai dari kelengkapan assignment. Setiap paket dalam `Problem` harus muncul sekali dalam `State`, sedangkan ID paket dan truk yang tidak dikenal ditolak. Paket yang berada di luar tidak memakai ruang maupun kapasitas truk.

Validator membuat *occupancy grid* (grid keterisian) NumPy bagi setiap truk. Bentuk grid mengikuti (W_t, L_t, H_t), dengan sel kosong bernilai -1 dan sel terisi menyimpan indeks paket. Jika paket i ditempatkan di truk t pada (x_i, y_i, z_i) dengan dimensi efektif (w_i, l_i, h_i), seluruh bagian paket harus memenuhi

$$\begin{aligned} 0 \le x_i, \quad x_i + w_i \le W_t, \\ 0 \le y_i, \quad y_i + l_i \le L_t, \\ 0 \le z_i, \quad z_i + h_i \le H_t. \end{aligned} \tag{2}$$

Dalam kode, `_collect_occupancies` membandingkan posisi dan dimensi paket dengan ukuran grid. Cuplikan 5 menunjukkan pemeriksaan paket yang melewati dinding peti kemas. Setelah lolos, irisan sel yang akan ditempati diperiksa lagi. Cuplikan 6 menolak paket ketika irisan tersebut sudah berisi indeks paket lain. Karena grid dibuat per truk, paket pada truk berbeda tidak saling bertabrakan.

Cuplikan 5. Pemeriksaan batas peti kemas

```
if any(
    start < 0 or start + size > limit
    for start, size, limit in zip(origin, dimensions, grid.shape, strict=True)
):
    errors.append(
        f"Package {placement.package_id} is out of bounds in truck {truck_id}"
    )
    unsafe_trucks.add(truck_id)
    continue
```

Paket pada $z_i = 0$ ditopang oleh lantai truk. Untuk paket di atas lantai, validator memeriksa sel tepat di bawah alasnya, yakni pada $z_i - 1$. Paket dinyatakan **melayang** jika tidak ada satu pun sel alas yang mendapat support dari paket lain. Jika ada support tetapi salah satu paket penopang bersifat rapuh, susunan juga ditolak. Pemeriksaan ini mengikuti definisi support pada tugas, yaitu cukup ada satu titik alas yang ditopang. Validator tidak mensyaratkan seluruh alas paket bersentuhan dengan penopang. Bagian inti pemeriksaan tersebut tampak pada Cuplikan 7.

Setelah posisi dan support diperiksa, validator menghitung beban setiap truk. Untuk truk t, jumlah berat paket yang ditugaskan kepadanya harus memenuhi

$$\sum_{i \in P_t} \text{weight}_i \le \text{max_capacity}_t. \tag{3}$$

Cuplikan 6. Pemeriksaan overlap pada occupancy grid

```
for placement, package_index, region in regions:
    truck_id = placement.truck_id
    assert truck_id is not None
    cells = occupancies[truck_id][region]
    if np.any(cells != -1):
        errors.append(
            f"Package {placement.package_id} overlaps another in truck {truck_id}"
        )
        unsafe_trucks.add(truck_id)
        continue
    cells[...] = package_index
```

Cuplikan 7. Pemeriksaan support dan paket rapuh

```
w, l, _ = get_oriented_dimensions(
    packages[placement.package_id], placement.orientation
)
x, y, z = position.x, position.y, position.z
below = occupancies[truck_id][x : x + w, y : y + l, z - 1]
supporters = np.unique(below[below >= 0])
if supporters.size == 0:
    support_errors.append(
        f"Package {placement.package_id} is floating in truck {truck_id}"
    )
elif any(problem.packages[int(index)].is_fragile for index in supporters):
    fragile_errors.append(
        f"Package {placement.package_id} rests on fragile support "
        f"in truck {truck_id}"
    )
```

Cuplikan 8 menunjukkan bahwa berat dijumlahkan dengan `Decimal`. Dengan cara ini, batas seperti $0.1 + 0.2 \leq 0.3$ tidak keliru akibat pembulatan bilangan *floating point*. Hasil `validate_state` berupa `ValidationResult` yang memuat status `valid`, daftar `errors`, dan total berat per truk. Jika assignment atau geometri sudah ambigu, pemeriksaan support dilewati karena grid tidak lagi dapat dipercaya, sementara kesalahan yang sudah ditemukan tetap dilaporkan.

Cuplikan 8. Penjumlahan berat dan pemeriksaan kapasitas

```
exact_weights = {truck.id: Decimal(0) for truck in problem.trucks}
for placement in loaded:
    assert placement.truck_id is not None
    exact_weights[placement.truck_id] += Decimal(
        str(packages[placement.package_id].weight)
    )
truck_weights = {
    truck_id: float(weight) for truck_id, weight in exact_weights.items()
}
for truck in problem.trucks:
    if exact_weights[truck.id] > Decimal(str(truck.max_capacity)):
        errors.append(
            f"Truck {truck.id} exceeds capacity: "
            f"{truck_weights[truck.id]} > {truck.max_capacity}"
        )
```

3.2.4 Objective Function

Setelah state dinyatakan sah, program menghitung nilainya melalui `ObjectiveMode` pada `app/optimization/objective.py`. **Mode utama**, `TOTAL_VALUE`, menjumlahkan `value` seluruh paket yang dimuat pada armada:

$$f_{\text{total}}(s) = \sum_{i:\,\text{truck}_i \neq \text{None}} \text{value}_i. \tag{4}$$

Mode tambahan, `ETA_WEIGHTED_VALUE`, memberi bobot lebih besar pada paket yang ETA-nya lebih dekat tanpa menambah atribut masukan:

$$f_{\text{ETA}}(s) = \sum_{i:\,\text{truck}_i \neq \text{None}} \frac{\text{value}_i}{\text{eta}_i + 1}. \tag{5}$$

Kontribusi setiap paket termuat dihitung oleh `objective_contributions`, lalu dijumlahkan oleh `evaluate_objective` dengan `math.fsum`. Evaluator tidak memberi penalti atas pelanggaran constraint dan tidak memeriksa kelayakan state. Dalam alur pencarian, `validate_state` memeriksa kelayakan sebelum skor digunakan, sehingga **skor hanya dibandingkan di antara state yang sah**.

3.3 Implementasi Algoritma *Local Search*

3.3.1 Inisialisasi State dan Pembangkitan Neighbor

Sebelum pencarian dimulai, `runner.py` membuat satu *random number generator* dari *seed* yang diterima. Generator yang sama diteruskan ke inisialisasi dan algoritma agar run dengan input serta seed yang sama dapat diulang. Hill climbing dan simulated annealing menerima satu initial state dari `generate_random_feasible_state`. Genetic algorithm memakai fungsi yang sama untuk membentuk anggota populasi awal.

Inisialisasi memulai semua paket di luar truk, lalu mengacak urutan paket dan tujuan penempatannya. Untuk setiap truk yang dicoba, fungsi memilih orientasi dan posisi. Koordinat x dan y diambil dari rentang yang masih muat, sedangkan z diambil dari lantai atau permukaan atas paket lain. Kandidat baru hanya disimpan bila lolos

Cuplikan 9. Pembentukan initial state dalam `initializer.py`

```
state = State(
    placements=[PackagePlacement(package_id=p.id) for p in problem.packages]
)
indices = list(range(len(problem.packages)))
rng.shuffle(indices)
```

`validate_state`. Paket dapat tetap di luar saat target luar terpilih atau ketika percobaan penempatan gagal. Cuplikan 9 menunjukkan titik awal konstruksi tersebut.

Setelah initial state tersedia, neighbor dibentuk lewat tiga move pada `neighbors.py`. `RELOCATE` memindahkan paket ke koordinat atau truk lain, termasuk ke luar truk. `ROTATE` mengubah orientasi paket termuat sambil mempertahankan titik posisinya. `SWAP` menukar tujuan truk dan posisi dua paket, sementara masing-masing mempertahankan orientasinya. Peluang pemilihan awalnya berturut-turut 0,50, 0,25, dan 0,25. Setiap move bekerja pada salinan state agar state asal tidak berubah.

Proposal move belum tentu memenuhi constraint. Untuk hill climbing dan simulated annealing, `generate_random_neighbor` mengulang proposal sampai menemukan state sah atau mencapai batas percobaan, seperti pada Cuplikan 10. Hill climbing dapat meminta beberapa neighbor berbeda melalui `sample_valid_neighbors`, sedangkan simulated annealing mengambil satu. Pada genetic algorithm, proposal mutasi yang belum sah boleh masuk ke tahap repair sebelum dinilai.

Cuplikan 10. Penerimaan neighbor yang sah dalam `neighbors.py`

```
for _ in range(config.max_attempts):
    candidate = propose_random_move(problem, state, rng, config)
    if candidate is not None and validate_state(problem, candidate.state).valid:
        return candidate
return None
```

3.3.2 Hill Climbing

Keempat varian hill climbing diimplementasikan dalam kelas `HillClimbing` pada `app/optimization/algorithms/hill_climbing.py`. Setiap iterasi mengambil sampel neighbor sah melalui `sample_valid_neighbors`, kemudian menghitung objective-nya. Jumlah sampel diatur oleh `candidates_per_iteration`, dengan nilai default 50. Pembangkitan dibatasi hingga jumlah kandidat dikali `max_attempts` percobaan, dengan default 30, sehingga sampel yang diperoleh dapat lebih sedikit dari permintaan. Perbedaan tiap varian terletak pada cara memilih kandidat dan melanjutkan pencarian. Cuplikan berikut merangkum bagian implementasi yang digunakan masing-masing varian, dengan pencatatan riwayat tidak ditampilkan.

Steepest-ascent hill climbing membandingkan skor state saat ini dengan seluruh neighbor sah yang berhasil masuk ke sampel iterasi. Secara teoretis, steepest-ascent memilih neighbor terbaik dari seluruh neighborhood; implementasi hanya dapat memilih yang terbaik dari sampel, yang jumlahnya dibatasi dan dapat kurang dari target bila proposal sah tidak cukup ditemukan. Jadi, `local_optimum` menandai tidak adanya peningkatan pada sampel, bukan bukti bahwa semua neighbor sudah diperiksa. Langkah yang lebih

baik dipilih, dan jika beberapa kandidat berbagi skor tertinggi, satu dipilih secara acak seperti pada Cuplikan 11. Aturan ini mengarahkan pencarian ke kenaikan tertinggi yang teramati, tetapi dapat berhenti di puncak lokal atau sebelum menemukan peningkat di luar sampel.

Cuplikan 11. Inti pemilihan neighbor pada steepest-ascent

```
top_score = max(scores)
if top_score <= current_score:
    termination_reason = "local_optimum"
    break
pool = [
    (n, score)
    for n, score in zip(neighbors, scores)
    if score == top_score
]
selected, selected_score = rng.choice(pool)
current, current_score = selected.state, selected_score
```

Stochastic hill climbing menyaring neighbor sah dalam sampel. Kandidat dengan skor yang meningkat ketat kemudian dipilih secara seragam. Peluang tersebut tidak dibobot menurut besar kenaikan: kandidat dengan peningkatan kecil memiliki peluang yang sama dengan kandidat yang lebih tinggi. Akibatnya, langkah terpilih bisa lebih rendah skornya daripada kandidat terbaik dalam sampel, tetapi tetap memperbaiki state saat ini. Cuplikan 12 memperlihatkan penyaringan dan pemilihan itu. Jika tidak ada kandidat peningkat, algoritma berhenti dengan alasan `local_optimum`; riwayat juga mencatat banyaknya kandidat peningkat yang ditemukan. Pilihan acak mengubah lintasan pencarian, tetapi aturan peningkatan ketat membuat varian ini tidak dapat melintasi plateau.

Cuplikan 12. Pemilihan acak dari neighbor peningkat pada stochastic

```
if is_stochastic:
    pool = [
        (n, score)
        for n, score in zip(neighbors, scores)
        if score > current_score
    ]
    if not pool:
        termination_reason = "local_optimum"
        break
    selected, selected_score = rng.choice(pool)
    current, current_score = selected.state, selected_score
```

Hill climbing with sideways move memilih kandidat dengan skor tertinggi dalam sampel seperti steepest-ascent, termasuk kandidat yang skornya sama dengan state saat ini. Langkah setara memungkinkan pencarian berpindah di plateau, tempat kenaikan mungkin belum terlihat dari posisi sekarang. Batas `max_sideways` berlaku untuk langkah setara berturut-turut: setiap langkah tersebut menambah penghitung, dan peningkatan skor mengatur ulang penghitung ke nol. Dengan batas default 100, pencarian dapat mengambil hingga 100 langkah setara berurutan; jika langkah setara berikutnya masih diperlukan, pencarian berhenti dengan alasan `max_sideways`. Cuplikan 13 memperlihatkan aturan tersebut. Kandidat yang menurunkan skor tetap ditolak, dan jumlah langkah sideways

dicatat. Batas ini mengendalikan penelusuran plateau, tetapi tidak menjamin bahwa plateau telah ditelusuri seluruhnya.

Cuplikan 13. Penerimaan dan penghitungan sideways move

```
if top_score > current_score or (
    top_score == current_score and consecutive_sideways < max_sideways
):
    pool = [
        (n, score)
        for n, score in zip(neighbors, scores)
        if score == top_score
    ]
    selected, selected_score = rng.choice(pool)
    improved = selected_score > current_score
    current, current_score = selected.state, selected_score
    if improved:
        consecutive_sideways = 0
    else:
        consecutive_sideways += 1
        sideways_moves += 1
```

Random-restart hill climbing memakai steepest-ascent untuk setiap climb, dimulai dari state awal runner lalu dilanjutkan dengan state sah baru pada tiap restart. Dengan 10 restart, implementasi melakukan 11 climb. Setiap climb mendapat batas `max_iterations` sendiri, sehingga anggaran itu berlaku per climb, bukan dibagi di antara semua restart. Cuplikan 14 memperlihatkan pengulangan dan pemilihan hasil: **state akhir** berasal dari climb terakhir, sedangkan `best_state` dan skor terbaik berasal dari climb dengan hasil tertinggi di seluruh pengulangan. Riwayat menggabungkan iterasi dengan penanda restart dan skor terbaik kumulatif. Restart memberi kesempatan untuk memulai dari wilayah lain dan mengurangi risiko bergantung pada satu puncak lokal, dengan biaya climb tambahan. Karena jumlahnya tetap terbatas dan tiap climb hanya memeriksa sampel neighborhood, metode ini tidak memastikan pencarian menemukan optimum global.

Cuplikan 14. Pengulangan steepest-ascent pada random restart

```
start = initial_state
for restart in range(config.max_restarts + 1):
    if restart > 0:
        start = generate_random_feasible_state(problem, rng)
    climb = self._climb(
        problem,
        start,
        variant=HillClimbingVariant.STEEPEST_ASCENT,
        objective_mode=objective_mode,
        rng=rng,
        config=config,
        include_state_history=include_state_history,
    )
    climbs.append(climb)
best = max(climbs, key=lambda climb: climb.best_score)
```

Setiap climb juga berhenti jika tidak ada neighbor sah (`no_valid_neighbor`) atau batas iterasi tercapai (`max_iterations`). State saat ini dan state terbaik disimpan ter-

pisah. Melalui API, pencarian dibatasi hingga 2.000 iterasi per climb, 50 kandidat per iterasi, dan 20 restart.

3.3.3 Simulated Annealing

Kelas `SimulatedAnnealing` memulai pencarian dari state sah yang sama jenisnya dengan hill climbing, tetapi hanya mengambil **satu neighbor acak** pada setiap iterasi. Move dengan $\Delta f \geq 0$ langsung diterima. Untuk $\Delta f < 0$, algoritma menggunakan peluang $\exp(\Delta f/T)$ sebagaimana dijelaskan pada Bab 2. Cuplikan 15 menunjukkan keputusan tersebut di dalam kode.

Cuplikan 15. Penerimaan move pada `simulated_annealing.py`

```
delta = candidate_score - current_score
acceptance_probability = (
    1.0 if delta >= 0 else math.exp(delta / temperature)
)
accepted = delta >= 0 or rng.random() < acceptance_probability
if accepted:
    current = candidate.state.model_copy(deep=True)
    current_score = candidate_score
```

Temperatur awal dapat diberikan sebagai parameter. Jika kosong, nilainya diambil dari rata-rata value paket dengan batas bawah 1. Setelah satu iterasi selesai, temperatur dikalikan `alpha`, yang bernilai antara nol dan satu. Pencarian berhenti saat temperatur berada di bawah `min_temperature`, batas iterasi tercapai, atau neighbor sah tidak ditemukan. `best_state` disimpan terpisah dari state saat ini, sebab simulated annealing dapat menerima move yang menurunkan skor. Riwayat iterasi mencatat temperatur, peluang penerimaan, keputusan move, dan skor. Parameter `stuck_patience` hanya menandai dan menghitung kejadian ketika skor terbaik tidak membaik cukup lama; parameter ini tidak menghentikan pencarian.

3.3.4 Genetic Algorithm

Kelas `GeneticAlgorithm` memulai pencarian dengan **populasi state sah**. Runner tidak memberikan initial state tunggal; algoritma membangkitkan populasi awal, lalu champion populasi tersebut dicatat sebagai `initial_state` dan `initial_score` pada hasil. Satu state menjadi satu individual dan objective-nya menjadi fitness. Pemilihan induk memakai `roulette_select`, sehingga fitness lebih tinggi mendapat peluang lebih besar. Jika semua fitness nol, induk dipilih seragam secara acak. `uniform_crossover` menyusun anak dengan mengambil placement setiap paket dari salah satu dari dua induk, masing-masing dengan peluang setengah.

Dalam setiap generasi, satu individual dengan fitness tertinggi disalin langsung sebagai *elite*. Individual lain dibentuk melalui selection, crossover sesuai `crossover_probability`, dan mutation sesuai `mutation_probability`. Mutation memilih salah satu dari relocate, rotate, atau swap. Sebelum masuk ke populasi berikutnya, anak diproses oleh `repair_state`, seperti terlihat pada Cuplikan 16.

Repair memulai dari state dengan semua paket di luar, lalu memproses paket termuat menurut koordinat z dan ID. Placement yang masih layak dipertahankan; paket yang gagal ditempatkan ulang dicoba pada orientasi, posisi, dan truk lain, dan tetap di luar

Cuplikan 16. Pembentukan generasi baru pada `genetic_algorithm.py`

```
next_population = [champion.model_copy(deep=True)]
while len(next_population) < config.population_size:
    parent_a = roulette_select(population, fitnesses, rng)
    parent_b = roulette_select(population, fitnesses, rng)
    if rng.random() < config.crossover_probability:
        child = uniform_crossover(problem, parent_a, parent_b, rng)
    else:
        child = rng.choice((parent_a, parent_b)).model_copy(deep=True)
    if rng.random() < config.mutation_probability:
        proposal = rng.choice(
            (propose_relocate, propose_rotate, propose_swap)
        )(problem, child, rng)
        if proposal is not None:
            child = proposal.state
    next_population.append(repair_state(problem, child, rng))
```

bila tidak ada penempatan sah. Setiap generasi mencatat maksimum dan rata-rata fitness populasi; riwayat dimulai pada generasi 0, sehingga memiliki $G + 1$ catatan untuk G generasi. Champion generasi terakhir menjadi state akhir, sementara state terbaik sepanjang run disimpan terpisah. Elitisme mempertahankan satu champion. Skema memakai default populasi 50, 100 generasi, peluang crossover 0,80, peluang mutasi 0,20, dan satu elite. Batas API ialah populasi 100 dan 200 generasi.

4 Eksperimen dan Analisis

4.1 Rancangan Eksperimen

Rancangan berikut belum dilaksanakan dan belum memuat hasil pengukuran. Satu run merujuk pada satu eksekusi algoritma untuk satu problem dengan satu seed.

4.1.1 Data Uji dan Lingkungan Eksperimen

Objektif utama adalah **`TOTAL_VALUE`**, yaitu jumlah nilai paket termuat tanpa pembobotan ETA. Dataset utama dirancang dengan satu truk `truck-a` berukuran $10 \times 8 \times 6$ dan kapasitas 100, serta sembilan paket fixture pada Tabel 1 dengan atribut asli. `truck-b` dan `truck-c` tidak disertakan. Satu problem JSON menjadi snapshot tetap seluruh run agar variasi hasil dapat dikaitkan dengan algoritma, seed, atau konfigurasi.

ID	Dimensi (w, l, h)	Nilai	Berat	Rapuh	ETA
P01	$(4, 3, 2)$	40	20	Tidak	2
P02	$(3, 3, 2)$	30	15	Tidak	1
P03	$(2, 2, 1)$	25	5	Ya	0
P04	$(2, 4, 3)$	35	18	Tidak	3
P05	$(3, 2, 2)$	20	10	Tidak	4
P06	$(1, 1, 1)$	8	2	Ya	1
P07	$(5, 4, 3)$	50	30	Tidak	2
P08	$(6, 6, 6)$	60	40	Tidak	5
P09	$(2, 2, 2)$	12	6	Tidak	0

Tabel 1. Data paket fixture untuk dataset utama yang diusulkan.

Jumlah nilai seluruh paket adalah 280, sehingga menjadi **batas atas objektif yang diketahui**, bukan nilai optimum yang telah dicapai. Eksperimen multi-truk dapat menjadi kajian tambahan, tetapi hasilnya dianalisis terpisah dari dataset utama. Dokumentasi versi sistem operasi, CPU, RAM, Python, dependensi, dan kode aplikasi mendukung reprodusibilitas serta interpretasi durasi. Seluruh run dirancang berlangsung berurutan pada perangkat dan lingkungan runtime yang sama tanpa perubahan kode atau konfigurasi lingkungan di tengah rangkaian.

4.1.2 Konfigurasi Parameter dan Pengulangan Eksperimen

Setiap konfigurasi memiliki **tiga pengulangan dengan seed 42, 123, dan 2026**. Rancangan utama mencakup empat varian HC masing-masing tiga run, tiga run SA, serta dua kelompok GA masing-masing sembilan run, sehingga jumlahnya **33 run per dataset**. Konfigurasi GA dengan populasi 50 dan 100 generasi muncul pada kedua kelompok dengan label berbeda; enam run itu tercakup dalam 18 run GA. Default UI berupa populasi 6 dan 5 generasi berbeda dari konfigurasi tetap pada Tabel 2.

Setiap hasil direncanakan memiliki label run unik dan ID yang mengikat seed, parameter, mode objektif, serta problem JSON dengan keluarannya. Bukti utama per run mencakup **state awal, state akhir beserta skor objektif akhir, grafik objektif, durasi, dan jumlah iterasi aktual**. State dan skor terbaik menjadi informasi tambahan yang dibedakan dari hasil akhir, terutama untuk SA dan random-restart. Grafik

Metode	Konfigurasi	Run
HC steepest-ascent	Maksimum 2.000 iterasi, 50 kandidat per iterasi; tanpa sideways atau restart.	3
HC stochastic	Maksimum 2.000 iterasi, 50 kandidat per iterasi; pemilihan peningkat seragam.	3
HC sideways	Maksimum 2.000 iterasi, 50 kandidat per iterasi, maksimum 100 sideways berturut-turut.	3
HC random-restart	Maksimum 10 restart; 11 climb, masing-masing maksimum 2.000 iterasi dan 50 kandidat per iterasi.	3
SA	$T_0 = 30$, $\alpha = 0{,}995$, $T_{\min} = 0{,}001$, maksimum 5.000 iterasi, ambang stagnasi 50 iterasi.	3
GA: variasi generasi	Populasi tetap 50; generasi 50, 100, dan 200.	9
GA: variasi populasi	Generasi tetap 100; populasi 20, 50, dan 100.	9

Tabel 2. Kontrol dan jumlah run yang direncanakan.

memakai indeks iterasi pada sumbu horizontal dan skor objektif pada sumbu vertikal. Nilai `runtime_ms` mencakup inisialisasi dan pencarian, tetapi tidak mencakup validasi akhir, penulisan berkas, komunikasi HTTP, maupun rendering.

Simulasi direncanakan melalui UI dengan pencatatan manual dan tangkapan layar sebagai bukti utama. Arsip JSON menjadi dokumentasi tambahan yang bersifat opsional.

Pengarsipan memakai `POST /api/experiments/run`. Dengan `persist=true`, hasil disimpan di `output/experiments/`. Snapshot riwayat disertakan melalui `include_state_history=true`. Backend memeriksa kelayakan state sebelum memberikan respons.

Preset batch `ga_mandatory` memuat 18 run GA dan menghasilkan JSON serta CSV saat persistensi aktif. Batch menyimpan state awal, akhir, dan terbaik tanpa snapshot per iterasi, sehingga tidak menyediakan replay spasial. Karena UI menggunakan `persist=false`, hasilnya tidak tersimpan permanen melalui antarmuka. Ekspor API dan salinan problem menjadi bahan pendukung opsional bagi pencatatan manual, bukan pengganti tangkapan layar.

4.2 Hasil Eksperimen dan Visualisasi Program

4.2.1 Hill Climbing

4.2.2 Simulated Annealing

4.2.3 Genetic Algorithm

Label kelompok	Populasi	Generasi	Seed
`generations`	50	50	42, 123, 2026
`generations`	50	100	42, 123, 2026
`generations`	50	200	42, 123, 2026
`population_size`	20	100	42, 123, 2026
`population_size`	50	100	42, 123, 2026
`population_size`	100	100	42, 123, 2026

Tabel 3. Enam konfigurasi GA, masing-masing dengan tiga pengulangan.

4.3 Analisis Eksperimen

4.3.1 Perbandingan Kualitas Solusi

4.3.2 Perbandingan Durasi Pencarian

4.3.3 Konsistensi Hasil Eksperimen

4.3.4 Pengaruh Parameter Genetic Algorithm

5 Kesimpulan dan Saran

5.1 Kesimpulan

5.2 Saran

6 Pembagian Tugas Tiap Anggota Kelompok

Nama Lengkap	NIM	Tugas/Kontribusi
Samuelson Dharmawan Tanuraharja	13524001	
Nicholas Wise Saragih Sumbayak	13524037	
Neswa Eka Anggara	13524136	

Tabel 4. Pembagian tugas anggota kelompok

Daftar Pustaka

[1] Tim Asisten Laboratorium Inteligensi Artifisial, "Spesifikasi Tugas Besar 1 IF3170 Inteligensi Artifisial 2026/2027," ver. 1.4, Institut Teknologi Bandung, Bandung, Indonesia, Sep. 2026.

[2] S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[3] Tim Pengajar IF3170 Inteligensi Artifisial, "Beyond Classical Search," lecture slides, Institut Teknologi Bandung, Bandung, Indonesia, 2026.

A Form Penggunaan Generative AI

B Lampiran Lainnya (Bila Ada)